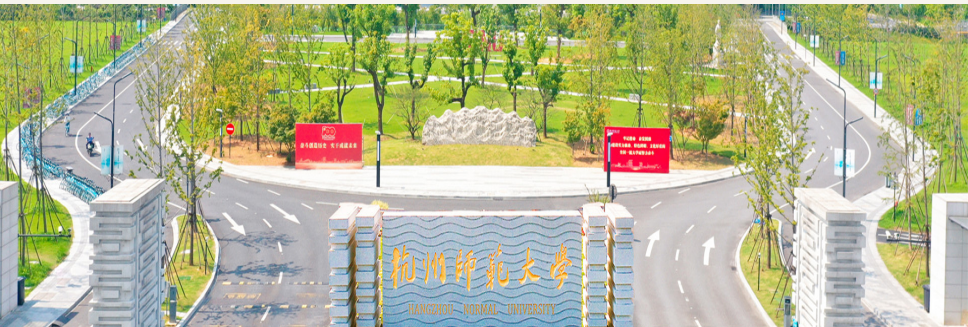


第六讲 - 向量语义与嵌入

张建章

阿里巴巴商学院
杭州师范大学

2026-03-01



- 1 向量语义模型的理论基础
- 2 词汇语义学
- 3 向量语义模型
- 4 余弦相似度
- 5 词频-逆文档频：在向量中为词语加权
- 6 点互信息
- 7 向量语义模型的应用
- 8 Word2vec
- 9 词嵌入可视化
- 10 词嵌入的语义属性
- 11 课后练习
- 12 第三次作业

- 1-2节：简要介绍词汇向量化表示的两个理论基础；
- 3-7节：理解稀疏的词语向量化表示方法、文档特征向量、文本向量相似度的内涵；
- 8-10节：以Word2Vec算法为例，理解稠密词语向量的构建原理，通过可视化和简单代数运算理解稠密词向量的蕴含的语义属性；

分布假设：词语的意义并非孤立存在，而是与它在文本中出现的环境密切相关。早在1950年代，Joos、Harris、Firth 等语言学家便观察到：**相近语义的词往往会在类似的上下文中共同出现**。这一观察形成了“**分布假设**”，即“词语相似性可以通过其分布特点来度量”。

向量表示：心理学家 Osgood 等人在 1957 年提出，将词语的情感语义信息用多维空间中的一个点来表示，提出了三个关键维度：**愉悦度 (valence)**、**唤起度 (arousal)** 和 **支配度 (dominance)**，每个词可以用一个三维向量来进行描述，从而**刻画其在情感上的位置**。

稀疏表示到密集嵌入的转变：

① 早期，基于共现计数的稀疏高维向量表示方法，如 tf-idf 和 PPMI 等，用高维稀疏向量直接刻画分布假设中的上下文；

② 现在，基于自监督学习的低维密集型词向量模型，如 word2vec、BERT embedding等，用低维稠密向量间接刻画分布假设中的上下文。

稀疏的含义是指向量中存在大量0元素。稀疏表示虽然**直观**，但**维数庞大且难以捕捉深层语义结构**；而密集嵌入（如 word2vec、GloVe）则能在**较低维空间中更有效地反映词语间的语义距离和相似性**。

1. 同义关系 (Synonymy)

当两个词在某个语义层面具有相似或近似的意义时，便可称其为同义词。同义的形式化定义为：若两个词可以在任意句子中相互替换而不改变句子的真值条件，则它们为同义词。由于形式上的差别必然伴随某种语义差异，在实际中很难找到绝对意义完全相同的词，因此同义关系通常指近似同义。

2. 词语相似性 (Word Similarity)

词语并不总是严格的同义，而更多是具有相似性。以“cat”与“dog”为例，它们在具体语义上并非替换性同义，但在人们的直观感知中仍被视为语义上相近。这种相似性为句子或文本的整体语义比较提供了依据。

3. 词语关联性 (Word Relatedness)

除了相似性外，词语之间还存在其他类型的关联性。例如“coffee”与“cup”虽然在属性上没有太多重合，却在实际使用中密切相关，因为它们常共同出现在饮用咖啡的情境中，属于同一个语义场。

语义场 (semantic field) 是指覆盖特定语义范畴、且彼此之间存在结构化关联的一组词汇，如外科医生、手术刀、护士等均属于医院相关词汇。语义场为发现文本中的主题结构和关联关系提供了理论支撑。

文档与词汇的向量空间模型

将文档和词汇映射到高维向量空间中的核心思想是：利用**共现矩阵**构建向量表示，以捕捉文本中蕴含的上下文信息（统计信息与语义联系）。

1. 利用文档构造向量（Term-Document Matrix）

在文档向量表示中，每一行对应词汇表中的一个词，每一列对应文档集合中的一个文档。矩阵中每个单元格记录了某一词在某一文档中出现的次数。

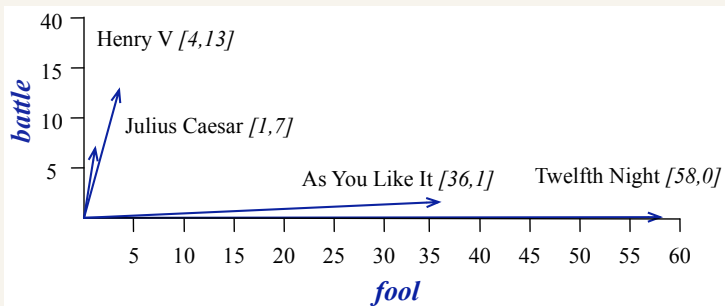
	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

3. 向量语义模型

每个文档可以看作一个向量，其维度等于词汇表大小。在此模型中，文档间的相似性通过比较对应向量（如在“battle”与“fool”这两个维度上的分布）得以体现。如果两个文档在大多数词汇上具有类似的计数分布，那么它们在向量空间中方向相近，表示语义上具有相似性。

下图中，两部莎士比亚的喜剧均在*fool*这个维度上取值高，在*battle*这个维度上取值低，而两部悲剧均在*battle*这个维度取值高，在*fool*这个维度上取值低。直观上可以看到两部悲剧（或两部喜剧）之间的夹角更小；



以文档为维度构造词向量

1. 词向量的构造

利用上述的词—文档矩阵，每个词可以用一行向量表示，其维度是文档数量。即一个词的向量反映了该词在不同文档中出现的频率分布。

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.5 The term-document matrix for four words in four Shakespeare plays. The red boxes show that each word is represented as a row vector of length four.

基于分布假设，相似词往往出现在相似的上下文（文档）中，因此其向量在文档维度下会表现出相近的数值分布。通过比较这些行向量，可以捕获词汇之间的语义相似性。这种方法直接利用文档统计信息来刻画每个词的语义特征，从而使得相近意义的词（尽管形式不同）在向量空间中趋于聚集（数据点聚集，向量夹角小），便于在后续任务中进行相似性度量和信息检索。

以词为维度构造词向量——词—词共现矩阵

1. 词—词 (Term-Term 或 Word-Word) 矩阵

除了使用文档作为向量维度 (粗粒度), 还可以利用词在上下文窗口内的共现情况来构造向量 (细粒度)。每一行仍表示目标词, 每一列对应的不是整个文档, 而是语境中的另一个词。统计目标词与各上下文词在固定窗口 (例如左右各4个词) 中的共现次数, 形成的便是词—词共现矩阵。由于词汇量通常较大 (例如常见词可能达到一万个甚至五万个), 因此这种矩阵往往是高维且稀疏的。

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 6.6 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser.

例如, “cherry” 与 “strawberry” 的共现向量在与 “pie” 或 “sugar” 等词上的计数较高 (上下文相似), 体现了二者在饮食或甜点语境下的密切联系。

3. 向量语义模型

例如，“digital”与“information”在相关上下文中的共现频次表明了两者的语义相关性。下图中，两个词的向量在*computer*和*data*维度均取值较高，向量夹角小。

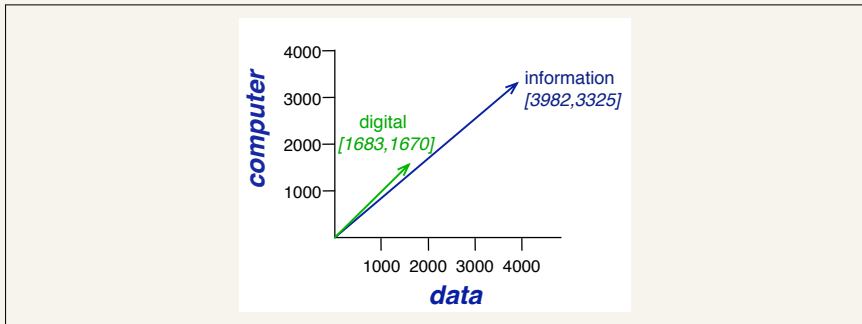


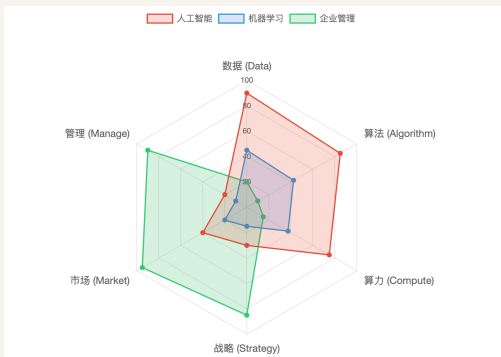
Figure 6.7 A spatial visualization of word vectors for *digital* and *information*, showing just two of the dimensions, corresponding to the words *data* and *computer*.

细粒度上下文的优势：利用词—词共现矩阵，可以更直接地反映词汇在局部语境中的相似性。与文档级别的统计不同，局部共现更能揭示词与词之间微妙的语义差异，对于捕捉同义、相似以及联想关系均具有较高的敏感度。

余弦相似度：从语义分布到几何夹角

1. 直觉逻辑：分布相似 $\Rightarrow$ 方向接近

- 词语相似意味着它们在文本中的分布规律相似，分布的“形状”相似即各维度上占比类似（可视化交互）。在向量空间中，这种分布相似性体现为两个向量的方向非常接近，即**夹角 θ 极小**。
- 夹角越小，其余弦值 $\cos(\theta)$ 越大（趋近于1），因此用余弦值来定义相似度。



2. 余弦相似度的定义

对于向量 $\mathbf{v}$ 与 $\mathbf{w}$ ，其余弦相似度定义为：

$$\cos(\theta) = \frac{\mathbf{v} \cdot \mathbf{w}}{|\mathbf{v}| |\mathbf{w}|} = \frac{\mathbf{v}}{|\mathbf{v}|} \cdot \frac{\mathbf{w}}{|\mathbf{w}|}$$

针对非负词频向量，余弦相似度取值范围为 $[0, 1]$ 。

3. 文本向量归一化：建立相似度与欧式距离的关系（可视化）

余弦相似度亦可理解为向量 $\mathbf{v}$ 与 $\mathbf{w}$ 模长归一化之后的点积，即 $\mathbf{v}_{norm} \cdot \mathbf{w}_{norm}$ 。对于单位向量 $\mathbf{v}_{norm}$ 和 $\mathbf{w}_{norm}$ ，其欧氏距离与余弦相似度有如下关系：

$$d(\mathbf{v}_{norm}, \mathbf{w}_{norm}) = \sqrt{2(1 - \cos \theta)}$$

这说明，对于归一化向量，欧氏距离的排序与余弦相似度的排序完全等价：余弦相似度越大（夹角越小），欧氏距离越小；反之亦然。

在文本挖掘应用中，通常对文本向量进行归一化操作，这样可以确保语义上相似的文本，在空间中不仅方向接近（夹角小）而且距离相近（聚集在一起）。

4. 理解点积是计算文本相似度的重要指标

- 对于归一化之后的文本向量：点积就是余弦相似度。

- 对于未归一化的文本向量（原始点积）：点积直观解释是计算特征重合量（对应维度取值乘积再求和）。文本向量的点积直接衡量了两个词“共享上下文特征的绝对重叠量”。当两个词在相同维度（共同上下文）上均有较高频次时，对应相乘并求和的结果就越大，这在统计上直观反映了它们语义特征的强重合度。原始点积表示的特征重合量，会受到模长的影响产生偏差，无法准确反映文本相似度。

例如，“数字化”、“智能化”、“项目”三个词语的向量分别为： $\mathbf{A} = [1, 2]$ ， $\mathbf{B} = [2, 4]$ ， $\mathbf{C} = [10, 1]$ ，两个维度分别表示词语在科技类文档和商业类文档中出现的频数。 $\mathbf{A} \cdot \mathbf{B} = 10$ ， $\mathbf{A} \cdot \mathbf{C} = 12$ ，受向量模长的影响，点积计算的结果未能准确反映词语的相似度，余弦相似度的计算结果为： $\cos(\mathbf{A}, \mathbf{B}) = 1$ ， $\cos(\mathbf{A}, \mathbf{C}) \approx 0.534$ 。

- 对于模长相似的文本向量：如果参与点积计算的文本向量模长相似，由 $\mathbf{v} \cdot \mathbf{w} = |\mathbf{v}| |\mathbf{w}| \cos(\theta)$ 可知， $|\mathbf{v}| |\mathbf{w}|$ 就相当于一个常数。此时，点积的大小就完全由 $\cos(\theta)$ 决定了，即，点积排序与余弦相似度排序结果相似，可以用点积刻画文本相似度。在大模型中，最经典的“缩放点积注意力（Scaled Dot-Product Attention）”就是基于此。Transformer大模型中，计算点积之前对词向量进行的层归一化操作（就是Z-Score标准化）将词向量各维度的取值缩放到了一个统一的数值范围内，使得词向量的模长相似。

余弦相似度计算示例

构造一个包含“cherry”、“digital”及“information”的共现计数矩阵，计算词语之间的余弦相似度：

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .018$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

上例结果直观地表明，“information”与“digital”在共现模式上具有高度相似性，而与“cherry”则相去甚远，从而体现出余弦相似度在捕捉词义相似关系上的有效性。

4. 余弦相似度

下图显示了三个词语在“pie”和“computer”两个维度构成的二维平面上的向量表示，词语越相似，向量的夹角越小。

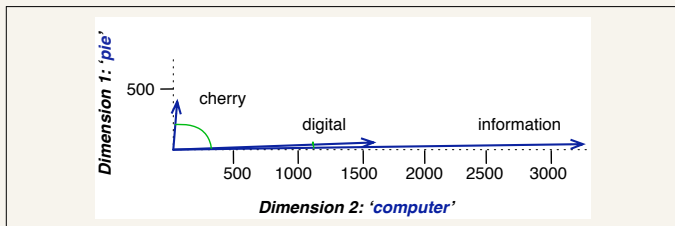


Figure 6.8 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

词相似性计算的应用场景：度量两个词的向量表示的相似性，进而在同义词发现、词义消歧、信息检索等任务中发挥重要作用。

文档相似度计算：通过计算文档向量间的余弦相似度，可以评估两个文档在主题、内容上是否接近，辅助分类、聚类等任务的开展。

问题背景及动机

频次统计的局限性：在构造词-文档共现矩阵时，通常直接使用原始频次来描述词语在文档中的出现情况。然而，原始频次分布往往呈现高度偏斜的特性：某些常见词（如“the”、“good”）在所有文档中均出现较频繁，这使得这些词在向量表示中占据较大的权重，导致其缺乏区分性。为了更好地捕捉对文档语义具有鉴别作用的单词，同时降低那些高频且信息量不高的单词的影响，就需要对原始频次进行“加权”处理。

权衡信息量与常见性的矛盾：一个单词在某文档中出现得越频繁，通常表明该单词对文档的主题具有较大贡献；但另一方面，如果一个单词在整个文档集合中均频繁出现，则它对区分各文档的重要性较低。

词频-逆文档频 (TF-IDF) 设计的初衷就是在“词频高代表重要”与“全局高频代表普遍性”之间取得平衡。

TF-IDF 的两个核心组成部分

TF-IDF 模型将单词在文档中的重要性拆解为两个相乘的部分：

1. 词频 (Term Frequency, tf)

词频 $tf_{t,d}$ 表示单词 t 在文档 d 中出现的次数。最简单的定义就是直接使用原始计数：

$$tf_{t,d} = \text{count}(t, d)$$

对频次的对数平滑处理：为避免词频极高的单词对模型产生过度影响，同时体现“出现 100 次并不意味着比出现 10 次重要 10 倍”这一直觉，通常采用对数变换对频次进行“平滑”：

$$tf_{t,d} = \begin{cases} 1 + \log_{10}(\text{count}(t, d)) & \text{if } \text{count}(t, d) > 0, \\ 0 & \text{otherwise.} \end{cases}$$

例如，当单词在文档中出现 1 次、10 次、100 次时，对应的 tf 分别为 1、2、3，从而弱化频次增加带来的线性影响。

逆文档频率 (Inverse Document Frequency, idf)

逆文档频率 idf_t 用于降低在整个文档集合中普遍出现的单词的权重：

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

其中 N 为文档总数， df_t 为包含单词 t 的文档数。当 $df_t = N$ 时， idf_t 取最低值 (0)，以反映这些普遍性高的单词在区分文档时信息量较低。

文档频率 df_t 表示单词 t 在多少个文档中出现；与之不同，集合频率指单词在所有文档中出现的总次数。即便两个单词在全集中出现的总次数相同，但如果一个单词只在极少数文档中出现，而另一个单词则几乎遍布所有文档，则前者能够更好地区分文档主题，应赋予更高权重。例如，莎士比亚全集中“Romeo”与“action”两词出现次数相同，但“Romeo”只在一部戏剧中出现，因此其 idf 值要高于“action”。

TF-IDF 权重的计算及示例

将词频与逆文档频率相乘，得到单词 t 在文档 d 中的 TF-IDF 权重：

$$w_{t,d} = tf_{t,d} \times idf_t$$

该公式同时考虑了单词在局部文档中的重要性与其在全局集合中的区分性，从而对各个单词赋予合适的权重。

计算实例：在37部莎士比亚戏剧中统计词语的逆文档频，如下：

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Figure 6.2 The term-document matrix for four words in four Shakespeare plays. Each cell contains the number of times the (row) word occurs in the (column) document.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Figure 6.9 A portion of the tf-idf weighted term-document matrix for four words in Shakespeare plays, showing a selection of 4 plays, using counts from Fig. 6.2. For example the 0.085 value for *wit* in *As You Like It* is the product of $tf = 1 + \log_{10}(20) = 2.301$ and $idf = .037$. Note that the idf weighting has eliminated the importance of the ubiquitous word *good* and vastly reduced the impact of the almost-ubiquitous word *fool*.

对于“good”这一单词，虽然在每个戏剧中均出现频繁，但由于它在所有戏剧中的文档频率很高，其 *idf* 值趋近于 0，因此最终的 TF-IDF 权重也很低，反映出其对区分不同戏剧主题信息量较小。同样，出现频率较低且局限于少数文档的单词（例如“Romeo”）会获得较高的 *idf* 值，从而使其 TF-IDF 权重较高，突显其在反映特定主题上的作用。

1. 基本理念

点互信息 (Pointwise Mutual Information, PMI) 用于判断两个词之间的“关联”程度，通过比较实际共现概率与独立假设下共现概率的比值，来定量衡量词与词之间的协同程度。

2. 数学定义

对于目标词 w 与上下文词 c ，PMI 定义为：

$$\text{PMI}(w, c) = \log_2 \frac{P(w, c)}{P(w) P(c)}$$

其中， $P(w, c)$ 表示在语料中 w 和 c 同时出现的概率（使用最大似然估计，从共现计数归一化得到）； $P(w)$ 与 $P(c)$ 分别是 w 与 c 各自的出现概率。

概率比值的含义：分子 $P(w, c)$ 反映了观察到的共现情况，分母 $P(w)P(c)$ 则是假设二者互相独立时的共现概率。取对数后可以得到一个数值，该值越大说明二者实际共现频率越超过独立假设下共现的概率，因而关联性越强。

3. 正点互信息 (PPMI)

PMI的取值范围理论上是从负无穷到正无穷，正值表示实际共现频率超过独立假设下的预期随机共现的概率，负值表示实际共现频率低于独立预期，但在实际语料中，负PMI值往往不具备可靠性且难以解释¹。此外，我们关注的是词语的共现而不是“互斥”。所以，采用PPMI (Positive PMI)，将所有负值置零。

$$\text{PPMI}(w, c) = \max \left(\log_2 \frac{P(w, c)}{P(w) P(c)}, 0 \right)$$

计算过程：首先构建词-词共现矩阵 F （行对应目标词，列对应上下文语境词）， f_{ij} 是词 i 与上下文语境 j 的共现频次，利用统计数据计算：

- 联合概率： $p_{ij} = \frac{f_{ij}}{\sum_{i,j} f_{ij}}$;
- 边缘概率： $p_{i*} = \sum_j \frac{f_{ij}}{\sum_{i,j} f_{ij}}$ 与 $p_{*j} = \sum_i \frac{f_{ij}}{\sum_{i,j} f_{ij}}$;
- 计算每个单元格的 PMI，并将小于零的值置为 0。

¹在通用语料库中，对于负相关（负 PMI 值），往往是因为语料库规模不够大，导致本该碰巧出现的词语没有出现。

计算示例

维基百科语料库中四个词语在五种上下文中的共现频次及边缘计数，为方便演示计算过程，在此假定不受其他词汇或上下文的影响。

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$P(w=\text{information}, c=\text{data}) = \frac{3982}{11716} = .3399$$

$$P(w=\text{information}) = \frac{7703}{11716} = .6575$$

$$P(c=\text{data}) = \frac{5673}{11716} = .4842$$

$$\text{PPMI}(\text{information}, \text{data}) = \log_2(.3399 / (.6575 * .4842)) = .0944$$

	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

Figure 6.11 Replacing the counts in Fig. 6.6 with joint probabilities, showing the marginals in the right column and the bottom row.

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

大多数0 PPMI值对应的原始PMI实为负数。例如， $PMI(\text{樱桃}, \text{计算机}) = -6.7$ ，这意味着在维基百科中“樱桃”和“计算机”共同出现的频率比独立假设下预期随机共同出现的概率还要低。此处的负值并不意味着这两个词语是负相关的（互斥，这两个词不能共同出现²），通常是因为语料库规模不够大，导致本该碰巧出现的词语没有出现。

²太喜欢这个耳机了，五星差评

实际计算中的共现矩阵构建

- **维度**: 在构建词-词共现矩阵 (Term-Term Matrix, 也称 Word-Word Matrix) 时, 行和列通常是相同的, 都对应词表 V , 矩阵的维度为 $|V| \times |V|$, 是一个沿着主对角线对称的**对称矩阵** (即, $f_{ij} = f_{ji}$);
- **共现窗口**: 计算词语共现频数时, 使用固定大小的滑动窗口。通常**选取目标词左右各 L 个词** (例如 ± 4 个词) 作为上下文语境;
- **单元格数值计算**: 所有概率的分母都是整个共现矩阵中所有单元格数值的总和 $N = \sum_{i,j} f_{ij}$, 虽然是对称矩阵, 但 **N 不存在重复计数**。因为其含义不是“文本中有多少个词汇对”, 而是滑动窗口算法在这个语料库中, 总共截取出了多少个“**目标-上下文**”观察实例。例如:

在文本“吃樱桃派”中, 使用 $L = 1$ 的共现窗口, “樱桃”和“派”只是挨在一起出现了一次, 但是滑动窗口算法, 把它切分成了**两个独立的方向性观测事件**:

① 当目标词是“樱桃”时, 算法生成了两个目标-上下文对:

(目标: 樱桃, 语境: 吃) $\rightarrow F[\text{“樱桃”}][\text{“吃”}] += 1$

(目标: 樱桃, 语境: 派) $\rightarrow F[\text{“樱桃”}][\text{“派”}] += 1$

② 当目标词是“派”时, 算法又生成了一个词对:

(目标: 派, 语境: 樱桃) $\rightarrow F[\text{“派”}][\text{“樱桃”}] += 1$

改进与平滑策略

PMI 指标倾向于对低频上下文词赋予非常高的值，因为低频事件下，即使共现次数较小，其比值 $\frac{P(w,c)}{P(w)P(c)}$ 也可能被放大，为了缓解对低频词的偏差，可以使用幂次调整上下文概率或者进行Laplace 平滑：

1. 幂次平滑 (α 参数)

$$P_{\alpha}(c) = \frac{\text{count}(c)^{\alpha}}{\sum_{c'} \text{count}(c')^{\alpha}}$$

通常取 $\alpha = 0.75$ 能取得较好的平衡效果。这样，对低频上下文词，其概率经过幂次平滑后增大，从而降低计算出的 PMI 值，避免极端偏高。

2. Laplace 平滑 (加 k 平滑)

另外，也可以通过在共现计数中加入一个小常数 k (例如 0.1 至 3 的范围内)，以达到平滑效果，从而防止零计数和降低低频词对结果的不合理影响。

TL;DR: TF-IDF矩阵是对“词汇-文档”共现频数矩阵的升级，PPMI矩阵是对“词语-词语”共现频数矩阵的升级。升级的共同点是：**剔除高频词的噪音影响**，找出真正能够反映文档特点的词，找出真正与目标词有强语义关联的词。升级的不同点：**TF-IDF 惩罚（IDF大）“跨文档的普遍性”**，**PPMI 惩罚（分母大）“跨语境的普遍性”**。

向量语义模型中，每个目标词都可由一个向量表示，其维度对应于整个文档集合（即词—文档矩阵）或局部上下文词（即词—词共现矩阵）。在这两种表示中，每一维的值均由词在特定文档中或在指定上下文中的出现次数得到，两种权重函数分别针对不同情形发挥作用：

TF-IDF: 在词-文档矩阵中，通过对局部词频进行对数平滑，再与逆文档频率的乘积，降低了在所有文档中高频出现的词的权重，**突出那些能区分文档特征的低频词**。

PPMI: 在词-词共现矩阵中，通过计算目标词与上下文词联合出现概率与独立出现概率比值的对数，并将负值置零，**使得实际共现显著高于预期的词对获得较高权重**，从而更好地揭示词汇间的语义关联。

语义比较：向量相似性计算

无论是 TF-IDF 还是 PPMI 加权后得到的向量，向量语义模型均采用余弦相似度作为两向量间相似性比较的主要指标。通过计算两个向量夹角的余弦值，能够衡量它们在语义空间中方向上的接近程度，余弦值越接近 1，说明两个词或文档在语义上越相似；反之，数值较低则表明二者差异较大。

文档向量的构造方法：将文档中所有词的向量进行求和后取平均，得到该文档的中心向量（质心）：

$$d = \frac{w_1 + w_2 + \cdots + w_k}{k}$$

这一操作实际是在多维空间中寻找一“平均点”，该点能最小化与文档中各词向量之间的平方距离总和。

TL;DR: 使用余弦相似度计算文本之间的相似度数，支持后续诸如文本聚类、文档检索等应用。

模型背景与动机

传统的文本表示方法（如基于词-文档或词-词共现矩阵的**稀疏向量表示**）虽然能够通过加权策略（TF-IDF、PPMI 等）捕捉词语之间的语义信息，但其**向量维度通常高达数万且稀疏，无法充分捕捉词汇之间的细微相似性**。

低维稠密向量（即词嵌入）通过将词语映射到一个紧凑的连续空间（通常仅为 50-1000 维），利用实数值来深度刻画语义特征。

这种表示方法不仅通过缩小参数空间显著提升了模型的泛化能力与计算效率，更核心的优势在于它能够捕捉词汇间的潜在关联，将语义相近的词汇映射到几何距离较近的位置，**弥补了稀疏表示在处理同义词时相互孤立的不足³**。

Word2vec 是通过**自监督训练**获取低维稠密词向量的典型代表。

³ 虽然 PPMI 能够通过共享上下文找到一定的相似度，但由于它将所有的上下文词都视为完全独立正交的维度，它无法识别上下文本身的同义性（如‘公路’与‘马路’的割裂），从而严重低估了目标词汇的细微相似性。而低维稠密向量通过空间的压缩，强制融合了同义上下文，跨越了这道语义鸿沟。

基本思想与模型直观

1. 嵌入向量的定义

Word2vec 的核心在于将词映射到一个低维连续向量空间中，这些向量通常维度在 50 到 1000 之间，与传统表示中基于词频的高维稀疏向量相比，具有更紧凑和更连续的特性。其每个维度虽没有明确的语义解释，但整体能够捕捉词的语义和句法信息。

2. 自监督学习范式

模型利用大量无标注文本中的上下文信息构造训练数据，无需人工标注。具体来说，通过利用“目标词与上下文词共现”这一自然现象，将该共现关系作为隐式监督信号，从而直接从数据中学习词向量表示。

模型构成: Skip-gram with Negative Sampling (SGNS)

1. 正、负样本构造

Word2vec 中最常用的实现方式为 Skip-gram 模型，其基本思想是：

① 将句子中**中心词**与其在一定窗口（例如 ± 2 个词）内的**上下文词**视为**正样本**。

② 针对每一个正样本，**随机从词汇中采样若干“噪声 (上下文)词”**，构建**负样本**，要求这些噪声词不等于目标词。

... lemon, a [tablespoon of apricot jam, a] pinch ...
 c1 c2 w c3 c4

This example has a target word w (apricot), and 4 context words in the $L = \pm 2$ window, resulting in 4 positive training instances (on the left below):

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

2. 基于嵌入相似度的概率计算

模型的核心假设是：若一个候选上下文词的嵌入向量与目标词的嵌入向量相似，则该词更有可能在目标词的上下文中出现。直观地，向量之间的相似性可以由它们的点积（内积）来衡量，即

$$\text{Similarity}(w, c) \approx c \cdot w$$

但由于点积值的范围是 $-\infty$ 到 $+\infty$ ，无法直接解释为概率。

3. 点积到概率的转换：使用 Sigmoid 函数

为了将点积转化为概率，模型使用 Logistic 回归中的 Sigmoid 函数：

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

正例概率：将目标词 w 与候选上下文 c 的点积经过 Sigmoid 转换后，得到

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

这一概率值介于 0 和 1 之间，高值表示 c 很可能为真实上下文词。

对应地，负例的概率定义为：

$$P(-|w, c) = 1 - P(+|w, c) = \sigma(-c \cdot w)$$

4. 整体分类器模型与多上下文处理

当目标词 w 拥有多个上下文词（记为 $c_1, c_2, \dots, c_L$ ）时，模型假设这些上下文词相互独立，则整个上下文窗口为正样本的联合概率（似然值）为：

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

其对数形式（对数似然）为

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

整体上，Skip-gram 模型训练一个**二元分类器**，其输入为目标词与候选上下文词对，通过计算二者嵌入向量的点积并经过 Sigmoid 转换，得到该候选词是否为真实上下文的概率。

模型参数：每个词的两种嵌入，① 作为目标词时的向量表示 W ，② 作为上下文词或噪音词时的向量表示 C 。

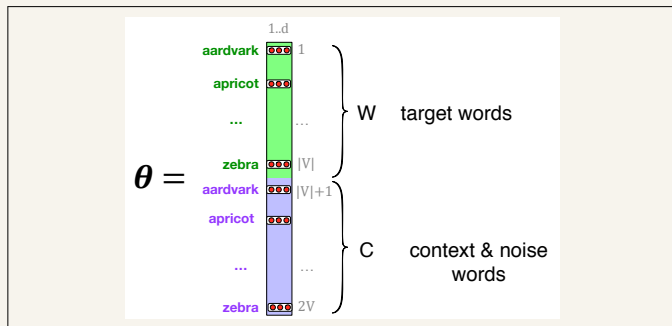


Figure 6.13 The embeddings learned by the skipgram model. The algorithm stores two embeddings for each word, the target embedding (sometimes called the input embedding) and the context embedding (sometimes called the output embedding). The parameter θ that the algorithm learns is thus a matrix of $2|V|$ vectors, each of dimension d , formed by concatenating two matrices, the target embeddings W and the context+noise embeddings C .

参数(词嵌入向量) 学习

1. 负样本构造

为了让模型不仅学会靠近真实上下文词，还能区分噪声词，每个正样本对应生成 k 个负样本对。负样本采用随机抽取方式，从词汇表中抽出不与目标词 w 相同的噪声词 c_{neg} 。这一步骤中，噪声词的采样概率并非简单的频率，而是按照加权的 **unigram 分布进行采样**⁴，其概率为

$$P_{\alpha}(w) = \frac{\text{count}(w)^{\alpha}}{\sum_{w'} \text{count}(w')^{\alpha}}$$

一般选择 $\alpha = 0.75$ ，这有助于提高低频词被采样的概率，从而减小频率极差带来的影响。

⁴ 计算每个词的频率，然后制作一个不均匀的多面体骰子，抛掷骰子，哪个词语朝上，就采样哪个词，加权就是对骰子进行打磨，避免高频词和低频词的极差太大，使得低频词没机会被采样到。

2. 训练目标与损失函数

为了使得模型能够区分“真实”上下文词和噪声词，构造如下目标函数。
 对于一个正样本 (w, c_{pos}) 及其 k 个负样本 (w, c_{neg_i}) ($i = 1, 2, \dots, k$)，定义整体损失函数为负对数似然：

$$L = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

其中，

- w 表示目标词的当前嵌入向量；
- c_{pos} 表示正样本中上下文词的向量；
- c_{neg_i} 表示第 i 个负采样噪声词的向量；
- $\sigma(x) = \frac{1}{1+\exp(-x)}$ 为 sigmoid 函数，其作用是将词向量间的点积转换为 $(0,1)$ 区间内的概率值。

这样设计的损失函数反映了两个目标：

- ① **最大化**正样本对 w 与 c_{pos} 的点积，使真实上下文词对的相似度更高；
- ② **最小化**目标词与负样本噪声词之间的相似度（最大化 $\log \sigma(-c_{neg_i} \cdot w)$ ），使非上下文词对不具备语义相关性；

3. 模型参数 (嵌入表示)

模型为每个词同时学习了两种嵌入：一种是作为“目标词”（输入嵌入）的向量 w ，另一种是作为“上下文词”（输出嵌入）的向量 c 。这两个向量通常分别存储在目标矩阵 W 和上下文矩阵 C 中。**这种双重表示的设计有助于模型在训练过程中更好地捕捉词与词之间的相关性。**

最终实际使用时，每个词语的嵌入向量表示，既可以将**两种嵌入相加**（即 $w + c$ ），也可以**只使用 W** 。

上下文窗口大小 L 的选取会对训练效果产生影响，通常需要根据具体任务进行在开发集上调参。

为最小化上述损失函数，采用**随机梯度下降**进行优化：计算损失函数对于两个嵌入向量的梯度，根据这些梯度对参数进行更新。

① 对正样本上下文词 c_{pos} 的梯度：

$$\frac{\partial L}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1] w$$

② 对每个负样本上下文词 c_{neg} 的梯度：

$$\frac{\partial L}{\partial c_{neg}} = \sigma(c_{neg} \cdot w) w$$

③ 对目标词 w 的梯度：

$$\frac{\partial L}{\partial w} = [\sigma(c_{pos} \cdot w) - 1] c_{pos} + \sum_{i=1}^k \sigma(c_{neg_i} \cdot w) c_{neg_i}$$

基于这些梯度，令学习率为 η ，可以得到参数更新公式：

① 正样本上下文词更新：

$$c_{pos} \leftarrow c_{pos} - \eta [\sigma(c_{pos} \cdot w) - 1] w$$

② 负样本上下文词更新：

$$c_{neg} \leftarrow c_{neg} - \eta \sigma(c_{neg} \cdot w) w$$

③ 目标词更新：

$$w \leftarrow w - \eta \left([\sigma(c_{pos} \cdot w) - 1] c_{pos} + \sum_{i=1}^k \sigma(c_{neg_i} \cdot w) c_{neg_i} \right)$$

算法首先以随机初始化的目标矩阵 W 和上下文矩阵 C 作为起点，然后遍历训练语料⁵，采用梯度下降法对 W 和 C 进行调整。

⁵ 滑动窗口读取目标词 -> 构造 1 正 k 负的样本组 -> 针对这组样本执行 SGD 更新参数。

word2vec的另一种模型：CBOW

- Skip-gram(一猜多):看到中心词“苹果”,预测周围可能出现“我”、“吃”、“很”、“甜”;
- CBOW(多猜一):看到上下文“我”、“吃”、“_____”、“很”、“甜”,预测中间空缺的那个词是“苹果”。

1. CBOW 算法的流程

假设当前中心词为 w_t , 其上下文窗口大小为 k , 则上下文词集合为 $Context(w_t) = \{w_{t-k}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+k}\}$ 。

① 词向量提取: 从输入矩阵 W 中获取所有上下文词的向量表示 w_i 。

② 向量聚合: 将所有上下文词的向量进行累加并取平均, 得到一个综合的上下文表示向量 $\hat{w}_t$:

$$\hat{w}_t = \frac{1}{2k} \sum_{-k \leq j \leq k, j \neq 0} w_{t+j}$$

注: 这里体现了“词袋”性质, 即 j 的顺序不影响 $\hat{w}_t$ 的结果。

③ 目标词预测: 使用聚合后的 $\hat{w}_t$ 与目标词 c_t (目标词 w 在 C 中的表示⁶中拿) 进行点击计算, 并使用sigmoid函数转换为概率。

⁶ 被预测的词, 其向量表示从 C

为了高效训练，同样引入负采样。对于目标词 c_t （正样本，对应向量为 c_{pos} ）和通过加权 Unigram 分布随机选取的 k 个噪声词（负样本，对应向量为 c_{neg_i} ），CBOW损失函数 L （负对数似然）定义如下：

$$L = - \left[\log \sigma(c_{pos} \cdot \hat{w}_t) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot \hat{w}_t) \right]$$

- $\hat{w}_t$: 输入的上下文平均向量（由 W 矩阵得出）；
- c_{pos} : 真实中心词在输出矩阵 C 中的向量；
- c_{neg_i} : 第 i 个噪声词在输出矩阵 C 中的向量；
- $\sigma(x)$: 激活函数 $\frac{1}{1+e^{-x}}$ ，将内积映射为概率。

CBOW 与 Skip-gram 的对比

CBOW 的优势（快而平稳）：因为它在一个滑动窗口中只更新一次梯度，计算量小很多⁷，所以训练速度快。同时，求平均的操作起到了平滑的作用，它对整体语义的把握比较好，对高频词的表示非常精确。

Skip-gram 的优势（慢而细腻）：它在一个滑动窗口中更新 $2k$ 次梯度，计算开销大，训练慢。但正因为每个词都与多个上下文词组成了多组训练样本，它对低频词、生僻词非常敏感和友好，能捕捉到更底层、更细腻的语义特征。在绝大多数现代 NLP 任务中，更倾向于默认使用 Skip-gram 模型训练词向量。

⁷ Skip-gram 在一个窗口中，有 $2k$ 个上下文词，构建了 $2k$ 组样本，使用随机梯度下降更新 $2k$ 次梯度。

其他类型的词嵌入向量

除word2vec之外，还有其他类型的静态词嵌入模型，这些模型虽然均生成**固定、不随上下文变化的词向量**，但在处理词汇覆盖和词形稀疏性等问题上存在不同的改进策略。

1. fastText 模型

(1) fastText 是 word2vec 的一种扩展，其主要改进在于解决未登录词 (OOV) (在测试语料中出现但在训练中未见到过的词) 和词形稀疏性问题⁸。

(2) 具体做法是利用子词建模，即不仅将词视作一个整体，同时将其分解为一系列字符 n-gram，并在每个词向量表示中加入这些 n-gram 的信息。

(3) 例如，对于单词“where”，模型会在其左右加上特殊边界符号形成“<where>”，再从中提取长度3-gram，即，“<wh”，“whe”，“her”，“ere”，“re>”。

(4) 在训练过程中，会为每个 n-gram 学习一个嵌入；最终，一个词的向量由该词自身与其所有 n-gram 嵌入的加和构成，从而不仅可以表示训练中见过的词，还可以对未知词进行建模，**对于未知词，就用未知词的n-gram对应的词向量求和表示**。

⁸ 社交媒体上的一些不规范词，如，good；丰富的词汇形态变化，如，run、runs，通过n-gram分解，稀疏词形与非稀疏词形会共享很多n-gram，从而弥补了词形稀疏词由于数据量不足而无法被学得准确的向量表示。

2. GloVe 模型

(1) GloVe (Global Vectors) 是另一种被广泛使用的静态词嵌入模型。

(2) 其主要思想在于利用全局共现统计信息（即基于词与词之间共现概率的比值）来捕获词的语义关系。

(3) GloVe 模型既结合了类似 PPMI 这类基于计数的方法的直观优势，又能像 word2vec 那样获得低维、连续的稠密向量表示。

(4) 数学上可以证明 word2vec 隐式地在优化某个与 PPMI 矩阵相关的目标函数，从而揭示了这两类方法之间的内在联系⁹。

⁹Levy, Omer, and Yoav Goldberg. "Neural word embedding as implicit matrix factorization." *Advances in neural information processing systems* 27 (2014).

1. 可视化的重要性

虽然嵌入向量通常处于高维空间（例如100维甚至更高），但为了直观理解词语之间的语义关系，需要将这些高维数据以易于感知（低维）的方式展示出来。有效地将高维向量映射到低维空间，是对词嵌入模型进行理解、应用和改进的重要工具。

2. 基于邻近词排序的简单可视化方法

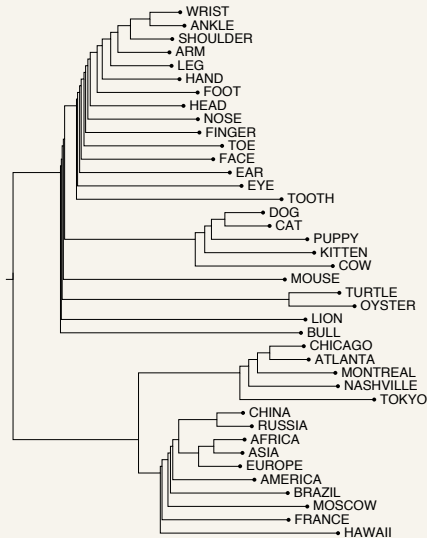
最直观的一种方法是：对给定的目标词 w ，通过计算其与词汇表中所有其他词向量的余弦相似度，排序后列出最相似的词。这种方法通过直接展示最近邻词，使得观察者能够快速捕捉到目标词在语义空间中的邻近分布情况。

3. 基于聚类分析的层次化可视化方法

另一种方法是利用聚类算法，将嵌入向量进行分层聚类，以生成一幅层次结构图。通过这种方式，可以直观地看出哪些词在语义上更为接近，从而以树状或层次结构的方式展示不同类别的词汇之间的相互关联。

9. 词嵌入可视化

下图展示了对若干名词进行层次聚类得到的树形图，反映出同属于某一语义类（如动物、地域、身体部位）的词在向量空间中的聚集趋势。



4. 低维投影技术的应用

目前最为常用的可视化技术是利用降维方法将高维嵌入向量投影到二维平面上。通过这种投影，不仅能够保留词向量之间的局部邻近关系，还能揭示出整体的聚类结构和全局语义布局。下图采用了t-SNE方法，将原本维度较高的嵌入向量降到二维，从而直观展示了词与词之间的相似性和分布特征。



Figure 6.1 A two-dimensional (t-SNE) projection of embeddings for some words and phrases, showing that words with similar meanings are nearby in space. The original 60-dimensional embeddings were trained for sentiment analysis. Simplified from [Li et al. \(2015\)](#) with colors added for explanation.

上下文窗口与词相似性的细分

1. 上下文窗口大小的影响

短窗口（例如 ± 2 词）：主要捕捉**局部语法信息及直接共现特征**，因此在最近邻排序中返回的词语通常在词性与具体意义上与目标词高度一致（即可替换性）。例如，目标词为“咖啡”，返回的往往是“茶”、“牛奶”、“果汁”。

长窗口（例如 ± 5 词）：捕获**更大范围内的主题或话题相关信息**，返回的词虽然在语法或词性上存在差异，但在语义或话题场景上具有较高**关联性**。例如，目标词为“咖啡”，返回的往往是“杯子”、“提神”、“咖啡馆”、“熬夜”。

2. 第一阶与第二阶共现

第一阶共现（Syntagmatic Association）：指两个词直接在**文本中相邻出现**的现象¹⁰，例如 *wrote* 常与 *book* 或 *poem* 共现，此类关系主要反映词的**搭配和语法结构**。**长窗口反映泛化的第一阶共现**。

第二阶共现（Paradigmatic Association）：关注的是**不同词在相似上下文环境中出现**¹¹，即虽然它们**不直接相邻**，仍表现出潜在的语义相似性。例如，*said*、*wrote*、*remarked*。**短窗口反映第二阶共现**。

¹⁰ word2vec 中来自 *C* 和 *W* 的向量求点积拟合的是第一阶共现。

¹¹ 求 *W* 中两个词向量的余弦相似度是拟合第二阶共现。

类比与关系相似性

1. 平行四边形模型

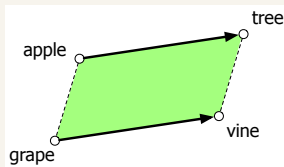
类比问题: *apple is to tree as grape is to vine.*

平行四边形模型: $\overrightarrow{tree} - \overrightarrow{apple} + \overrightarrow{grape} \approx \overrightarrow{vine}$, $\overrightarrow{tree} - \overrightarrow{apple}$ 计算出“苹果到树”的相对偏移量（也可以称为关系向量“__长在__”），把这个相对偏移量施加到 $\overrightarrow{grape}$ 上（把关系向量“__长在__”平移到 $\overrightarrow{grape}$ 上，即“grape 长在__”），就会得到 $\overrightarrow{vine}$ 。平行四边形模型本质上就是关系的平移。

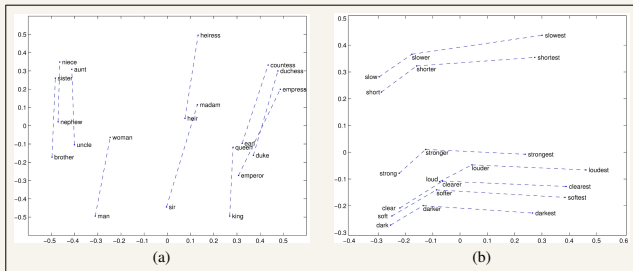
平行四边形模型数学上可表示为：

$$\hat{b}^* = \operatorname{argmin}_x \operatorname{distance}(x, b - a + a^*)$$

其中 a 、 b 和 a^* 分别对应类比问题中的已知词向量， $b - a$ 表示偏移量， $\hat{b}^*$ 为预测输出。



词嵌入向量模型能够应用于平行四边形模型，通过向量运算（如，**减法**）捕捉一些简单的词语之间关系信息。下图左侧显示了词向量对“男-女”性别关系（**偏移量**）的刻画，右侧为“比较级-最高级”关系。



上图中，GloVe 向量空间的关系属性（通过将向量投影至二维平面进行展示）。(a) $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}}$ 接近于 $\vec{\text{queen}}$ 。(b) 向量的偏移量（offsets）似乎捕捉到了比较级与最高级的词法形态特征。

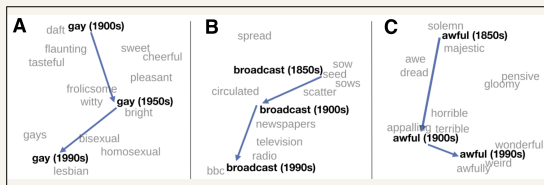
历史语义学的应用

1. 跨时代语义漂移分析

利用嵌入模型，可以在不同历史时期构建独立的语义空间，并比较同一词在各时间段的向量位置变化，从而揭示词义随时间的演变。下图展示了“broadcast”、“awful”等单词从旧义到新义的转变趋势。

2. 降维与可视化技术

通过使用 t-SNE 等降维方法，将各历史时期的高维嵌入向量映射到二维空间，直观展示词义变化的轨迹。



上图中，使用 word2vec 向量对 3 个英语单词的语义演变进行的 t-SNE 可视化。每个单词的现代词义以及灰色的上下文单词，是由最近（现代）时间点的嵌入空间计算得出的。较早的数据点则是从较早的历史嵌入空间中计算得出的。该可视化展示了：“gay”一词从与“快乐”或“嬉闹”相关的含义向指代同性恋的演变；“broadcast”一词从最初的播种之意向现代的“传输”之意的转变；以及“awful”一词的词义贬化过程，即从最初的“充满敬畏”转变为“可怕或糟糕”。

词嵌入中的偏见

① 嵌入模型在自动学习词义的同时，也不可避免地重现并放大了训练文本中存在的隐性偏见与社会刻板印象，这一现象在类比任务中表现尤为明显，并可能导致实际应用中的不公平配置，构成“配置伤害”。

例如，有研究 (Bolukbasi et al., 2016)发现¹²，在 word2vec 模型中，“computer programmer - man + woman”得到的最近邻词竟是“homemaker”；类似的，“father”与“doctor”的对应关系被模型映射为“mother”与“nurse”的相似性。

② 虽然已有研究尝试通过变换嵌入空间或修改训练策略来缓解这些偏见，但完全消除偏见仍然面临挑战。

③ 使用历史语料训练词嵌入提供了一种检测和分析语义偏见随时间演变的新视角。

¹² Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." *Advances in neural information processing systems* 29 (2016).

词嵌入评估

外部评估：强调词向量在实际应用场景中的效用，通过在具体 NLP 任务上的表现比较不同向量模型的效果；

内部评估：侧重于直接测量**向量之间的相似度与人类语义判断之间的相关性**，通过词相似性、同义词选择、上下文语义相似性以及类比等任务来直接度量模型捕捉语义关系的能力；

模型不稳定性与结果平均：由于嵌入模型存在固有的随机性和不稳定性，采用**多次训练并平均结果**¹³是一种必要的评估策略。

¹³ 例如，word2vec中两个词向量矩阵 W 和 C 就是随机初始化的。

1. word2vec的损失函数本质上是多个二分类交叉熵的和

以skip-gram模型为例，词对 (w, c) 为正样本（共现），则真实标签为 $y = 1$ ， (w, c) 为负样本（不共现），则真实标签为 $y = -1$ ，预测值 $\hat{y} = \sigma(w \cdot c)$ 。

首先，回忆一下二分类逻辑回归的交叉熵损失函数：

$$Loss_{BCE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

对于正样本，真实标签 $y = 1$ 。代入 BCE 公式：

$$\begin{aligned} Loss_{pos} &= -[1 \cdot \log(\hat{y}) + (1 - 1) \log(1 - \hat{y})] \\ &= -\log(\hat{y}) \\ &= -\log \sigma(c_{pos} \cdot w) \end{aligned}$$

对于负样本，真实标签 $y = 0$ 。代入 BCE 公式：

$$\begin{aligned} Loss_{neg} &= -[0 \cdot \log(\hat{y}) + (1 - 0) \log(1 - \hat{y})] \\ &= -\log(1 - \hat{y}) \\ &= -\log(1 - \sigma(c_{neg} \cdot w)) \end{aligned}$$

利用Sigmoid 对称性质: $1 - \sigma(x) = \sigma(-x)$:

$$Loss_{neg} = -\log \sigma(-c_{neg} \cdot w)$$

对于一个目标词, 我们构造了 1 个正样本和 k 个负样本。既然它们被拆成了 $1 + k$ 个独立的二分类任务, 那么整体的损失就是把这 $1 + k$ 个二元交叉熵直接加起来:

$$L_{total} = Loss_{pos} + \sum_{i=1}^k Loss_{neg_i}$$

把 $Loss_{pos}$ 和 $Loss_{neg}$ 的结果代入上式, 提取出前面的负号, 得:

$$L = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

2. Skip-gram模型参数梯度的详细求解过程

1. 损失函数 L 关于正样本上下文词 c_{pos} 的偏导数

损失函数中，只有第一项包含 c_{pos} ，即 $L_{pos_part} = -\log \sigma(c_{pos} \cdot w)$ 。

$$\begin{aligned}\frac{\partial L}{\partial c_{pos}} &= \frac{\partial}{\partial c_{pos}} [-\log \sigma(c_{pos} \cdot w)] \\ &= -\frac{1}{\sigma(c_{pos} \cdot w)} \cdot \frac{\partial}{\partial c_{pos}} \sigma(c_{pos} \cdot w)\end{aligned}$$

应用 Sigmoid 函数的导数性质 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$:

$$\begin{aligned}\frac{\partial L}{\partial c_{pos}} &= -\frac{1}{\sigma(c_{pos} \cdot w)} \cdot [\sigma(c_{pos} \cdot w)(1 - \sigma(c_{pos} \cdot w))] \cdot \frac{\partial}{\partial c_{pos}} (c_{pos} \cdot w) \\ &= -(1 - \sigma(c_{pos} \cdot w)) \cdot \frac{\partial}{\partial c_{pos}} (c_{pos} \cdot w)\end{aligned}$$

由于 $c_{pos} \cdot w$ 是两个向量的内积，对 c_{pos} 求导就得到了 w :

$$\frac{\partial L}{\partial c_{pos}} = (\sigma(c_{pos} \cdot w) - 1) \cdot w$$

2. 损失函数 L 关于某一负样本上下文词 c_{neg} 的偏导数
 L 中只有连加符号中的对应项包含它：

$$L_{neg_part} = -\log \sigma(-c_{neg} \cdot w)$$

$$\begin{aligned} \frac{\partial L}{\partial c_{neg}} &= \frac{\partial}{\partial c_{neg}} [-\log \sigma(-c_{neg} \cdot w)] \\ &= -\frac{1}{\sigma(-c_{neg} \cdot w)} \cdot \frac{\partial}{\partial c_{neg}} \sigma(-c_{neg} \cdot w) \end{aligned}$$

应用 Sigmoid 导数公式 $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ (注意 x 对应 $-c_{neg} \cdot w$):

$$\begin{aligned} \frac{\partial L}{\partial c_{neg}} &= -\frac{1}{\sigma(-c_{neg} \cdot w)} \cdot [\sigma(-c_{neg} \cdot w)(1 - \sigma(-c_{neg} \cdot w))] \cdot \frac{\partial}{\partial c_{neg}} (-c_{neg} \cdot w) \\ &= -(1 - \sigma(-c_{neg} \cdot w)) \cdot \frac{\partial}{\partial c_{neg}} (-c_{neg} \cdot w) \end{aligned}$$

运用Sigmoid 对称性质： $1 - \sigma(-x) = \sigma(x)$ 。所以 $1 - \sigma(-c_{neg} \cdot w)$ 就可以直接替换为 $\sigma(c_{neg} \cdot w)$ ：

$$\frac{\partial L}{\partial c_{neg}} = -\sigma(c_{neg} \cdot w) \cdot \frac{\partial}{\partial c_{neg}}(-c_{neg} \cdot w)$$

最后处理内积求导， $\frac{\partial}{\partial c_{neg}}(-c_{neg} \cdot w) = -w$ ：

$$\begin{aligned} \frac{\partial L}{\partial c_{neg}} &= -\sigma(c_{neg} \cdot w) \cdot (-w) \\ &= \sigma(c_{neg} \cdot w) \cdot w \end{aligned}$$

3. 损失函数 L 关于目标词 w 的偏导数

w 同时与正样本和所有的负样本发生了内积计算，所以它存在于损失函数的每一项中。为清晰起见，将求导过程分为两部分：对正样本部分的求导和对负样本部分的求导。

① 正样本项对 w 的偏导数（过程同 $\frac{\partial L}{\partial c_{pos}}$ ）：

$$\begin{aligned}\frac{\partial L_{pos}}{\partial w} &= -(1 - \sigma(c_{pos} \cdot w)) \cdot c_{pos} \\ &= [\sigma(c_{pos} \cdot w) - 1] c_{pos}\end{aligned}$$

② 负样本项对 w 的偏导数：

求和符号里的第 i 个负样本项对 w 的偏导数，过程同 $\frac{\partial L}{\partial c_{neg}}$ ：

$$\begin{aligned}\frac{\partial L_{neg_i}}{\partial w} &= -\sigma(c_{neg_i} \cdot w) \cdot (-c_{neg_i}) \\ &= \sigma(c_{neg_i} \cdot w) c_{neg_i}\end{aligned}$$

这只是第 i 个负样本的偏导，所有的 k 个负样本累加起来就是：

$$\sum_{i=1}^k \sigma(c_{neg_i} \cdot w) c_{neg_i}$$

将正样本的偏导数和所有负样本的偏导数加在一起，就得到了完整的 w 的梯度公式：

$$\frac{\partial L}{\partial w} = [\sigma(c_{pos} \cdot w) - 1] c_{pos} + \sum_{i=1}^k \sigma(c_{neg_i} \cdot w) c_{neg_i}$$

从稀疏表示到稠密嵌入的语义空间探索

实验目的

- 在真实语料库上构建**稀疏词向量**（TF-IDF）与**稠密词向量**（Word2Vec）
- 利用余弦相似度计算词汇间的语义距离
- 验证稠密向量在“语义类比”和“社会偏见”方面的属性

数据集准备

使用 **text8** 语料库（维基百科文本干净子集，约31MB）

```
import gensim.downloader as api
# 首次运行自动下载并缓存
dataset = api.load("text8")
```

- 无需手动下载复杂压缩包
- 首次运行自动下载并缓存
- 广泛用于词嵌入的入门训练

任务1: 基于稀疏表示的词语相似度 (TF-IDF)

- ❶ 截取 text8 前1000个文档, 使用 `scikit-learn` 的 `TfidfVectorizer` 构建词-文档矩阵, 降低全局高频词的权重
- ❷ 从词汇表中选取3组词对 (具有商业或技术背景), 例如 (只是例如):
 - "market" 与 "sales"
 - "computer" 与 "data"
 - "apple" 与 "cherry"
- ❸ 计算三组词对之间的余弦相似度:

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$

任务2：训练 Word2Vec 稠密词嵌入模型

- ① 使用 `gensim.models.Word2Vec`，在完整 `text8` 数据集上训练稠密词向量模型
- ② 参数要求：
 - 采用 **Skip-gram** 架构（设置 `sg=1`）
 - 引入**负采样**（`Negative Sampling`）
 - 使用**短上下文窗口**（如 `window=2`），捕捉词法替换性和直接共现特征
- ③ 提取**任务1中相同的三组词对**，利用稠密模型重新计算余弦相似度，与TF-IDF结果进行对比

任务3：探索语义属性——类比推理与潜在偏见

（一）平行四边形类比推理

利用 Word2Vec 模型复现关系偏移公式：

$$\vec{b}^* = \arg \min_x \text{distance}(x, b - a + a^*)$$

- 自行设计一组类比并进行验证，例如（只是例如），`king - man + woman`，`CEO - company + country`

（二）偏见观察

计算以下词向量偏移，观察前几个最近邻词汇：

- `programmer - man + woman`
- `doctor - man + woman`

验证模型中是否存在性别或社会刻板印象偏见（基于你所用的语料，有就有，没有就没有）。

实验结果分析

完成编码后，请结合运行结果回答以下问题：

问题1：对比分析

对比任务1（TF-IDF）和任务2（Word2Vec）中计算的词对余弦相似度：

- 你观察到了什么现象？
- 为什么稠密向量在捕捉语义关联词时通常表现更好？

问题2：现象剖析

针对任务3观察到的偏见现象，从商业数据分析师角度思考：

- 若将带有偏见的词向量模型用于企业**人力资源简历筛选系统**，可能造成怎样的“配置伤害”？
- 若用于**商品推荐系统**，又会带来哪些风险？

提交说明

提交格式

请提交 `.ipynb` (Jupyter Notebook) 文件

文件内容要求：

- 完整的可运行代码
- 完整的运行结果输出
- 对应的 Markdown 文本分析段落
- 每一步代码添加清晰注释，说明操作意图

注意

代码须能在标准 Python 环境下完整运行，缺少运行结果的提交不予评分。

THE END